A Robust Oversampling Approach for Class Imbalance Problem With Small Disjuncts

Yi Sun, Lijun Cai, Bo Liao, Wen Zhu, and Junlin Xu

Abstract—Class imbalance is one of the important challenges for machine learning because of it's learning to bias toward the majority classes. The oversampling method is a fundamental imbalance-learning technique with many real-world applications. However, when the small disjuncts problem occurs, how to effectively avoiding the negative oversampling results rather than using clusters previously, remains a challenging task. Thus, this study introduces a disjuncts-robust oversampling (DROS) method. The novel method shows that the data filling of new synthetic samples to the minority class areas in data space can be thought of as the searchlight illuminating with light cones to the restricted areas in real life. In the first step, DROS computes a series of light-cone structures that is first started from the inner minority class area, then passes through the boundary minority class area, last is stopped by the majority class area. In the second step, DROS generates new synthetic samples in those light-cone structures. Experiments considering both real-world and 2D emulational datasets demonstrate that our method outperforms the current state-of-the-art oversampling methods and suggest that our method is able to deal with the small disjuncts.

Index Terms—Imbalance problem, oversampling, small disjuncts, area illumination

1 Introduction

Class imbalance occurs when the number of samples in a given class (or classes) is smaller than the number of samples in other class (or classes), respectively called as minority class(es) and majority class(es). The class imbalance problem exists in many real-world applications like credit fraud detection [1], [2] steganography detection [3]. For example, given the training dataset consisting of 10 minority class samples and 90 majority class samples, the related classifier can achieve the 90% accuracy even if it identifies all samples as the majority class.

Over the years, class-imbalance learning methods can be roughly categorized into two groups, respectively as algorithm-level methods [4], [5], [6] and data-level methods [7], [8], [9]. The typical algorithm-level methods are the cost-sensitive learning techniques [3], [10] and ensemble learning techniques [11], [12]. The cost-sensitive learning techniques generally assign higher penalties to mis-classified minority class samples. For example, Castro *et al.* [13] proposed a cost-sensitive multilayer perceptron (MLP) to distinguish the importance of class errors. Zong *et al.* [14] proposed a weighted extreme learning machine (ELM) to maintain the advantages from original ELM. Zhang *et al.* [15] proposed a cost-sensitive deep brief network to optimize the mis-classification costs. Wu *et al.* [16] proposed an uncorrelated cost-sensitive multiset learning (UCML) approach to provide a multiple sets construction strategy. However, such methods tend to be specifically designed for certain applications and is thus not directly applicable on other applications [17]. Besides, the cost-sensitive learning techniques are also sensitive to noisy data points [18].

On the other hand, the ensemble learning techniques [11], [12] make use of Bagging and Boosting [19] for imbalanced data classification. For example, Bader-El-Den *et al.* [20] presented a novel ensemble-based method to increase the number of classifiers. Razavi-Far *et al.* [21] proposed the imputation-based ensemble techniques to train the ensemble base-learners. However, ensembles were not originally proposed to address the class-imbalance problems [17], and how to effectively guarantee and utilize the diversity of classification ensembles is still an open problem [16].

The typical data-level methods are the undersampling techniques [22], [23], [24] and oversampling techniques [25], [26], [27]. The undersampling techniques eliminate the majority class samples to balance the class distribution. For example, W. Y. Ng *et al.* [28] proposed a diversified sensitivity-based undersampling method by iteratively repeating the clustering and sampling to select a balanced set of samples. Lin *et al.* [29] introduced two undersampling strategies by using the cluster centers or their nearest neighbours to represent the majority class. Kang *et al.* [24] proposed a new undersampling scheme by incorporating a noise filter before resampling. However, such methods tend to loss the important information from the raw data.

On the other hand, the oversampling techniques fill the minority class areas with synthetic data points to balance the class distribution. The most straightforward oversampling technique is to repeat the minority class samples for several times. But it only takes the magnitude of class size into consideration that would lead to over-fitting.

- *Yi Sun, Lijun Cai, and Junlin Xu are with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China. E-mail: id.yisun@gmail.com, ljcai@hnu.edu.cn, 18273118685@163.com.*
- *Bo Liao and Wen Zhu are with the Key Laboratory of Computational Science and Application of Hainan Province, Hainan Normal University, Haikou 571158, China. E-mail: {dragonbw, syzhuwen}@163.com.*

Manuscript received 2 Apr. 2021; revised 4 Mar. 2022; accepted 18 Mar. 2022. Date of publication 23 Mar. 2022; date of current version 1 May 2023.
This work was supported by the National Key R&D Program of China under Grant 2020YFB2104400.
(Corresponding authors: Lijun Cai and Bo Liao.)
Recommended for acceptance by V. Moreira.
Digital Object Identifier no. 10.1109/TKDE.2022.3161291

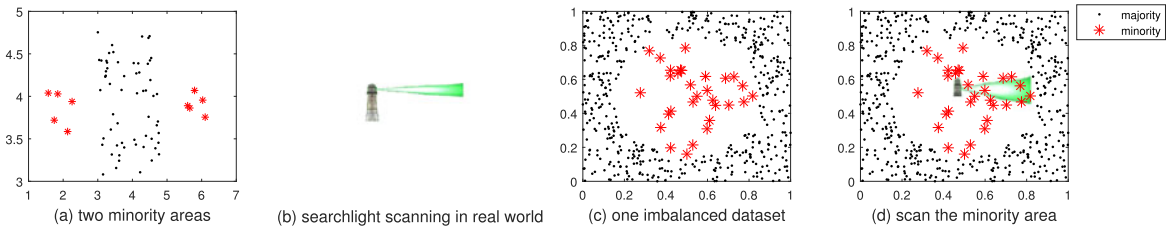


Fig. 1. Motivation of proposed method. (a) two minority class areas, (b) searchlight illuminating in real world, (c) an imbalanced dataset, (d) illuminate or scan the minority class area.

Thus, Chawla *et al*. [30] proposed a synthetic minority oversampling method (SMOTE) by linearly generating new samples between a seed minority class point and one of it's k nearest neighbours. However, dealing with the class imbalance problem becomes an even more challenging task in contexts with abnormal data distributions, including outliers, class overlapping or small disjuncts [31]).

In this study, we propose a disjuncts-robust oversampling (DROS) method for the small disjuncts problem, where a small disjunct is a disjunct that includes a few samples. This technique is based on the notion that each minority class area can be illuminated by light cones that surround its inner core zone rotationally, while launching these light cones from different inner core zones can implicitly help to separate the minority class areas. In this study, we refer to the light cone as the light-cone structure and the inner core zone as the inner minority class area. In detail, we respectively regard the minority class areas and the majority class areas in data space as the restricted areas and the barrier-building areas in real life. Thus, initially, DROS computes a series of light-cone structures that is first started from the corresponding inner minority class area, then passes through the boundary minority class area, last is stopped by (or projected on) the majority class area. Finally, DROS generates synthetic samples in those light-cone structures.

Main contributions can be summarized as follows.

1) We propose a novel disjuncts-robust oversampling method, that is of natural robustness to small disjuncts and good robustness to class overlapping and noisy data points.
2) We give a new relationship between a pair of minority class points.
3) We give the definition of light-cone structure in $\mathbb{R}^n$.

The paper is organized as follows. Section 2 reviews related works. And Section 3 introduces the novel oversampling technique. Section 4 shows the experimental results. Conclusion are included in Section 5.

2 Related Works

The interpolation-based techniques often generate new samples between the line-segment of a seed minority and one of it's neighbours. For examples, SMOTE [30] generates the new sample in the line-segment between a seed minority class point and one of it's k nearest neighbours. B-SMOTE [32], a recent extension of SMOTE, only selects minority class samples near to the borderline for data generation. Besides, ADASYN [33] assigns higher weights for minority class samples surrounding with larger number of majority in it's k nearest neighbours. However, the interpolation-based techniques tend to generate noisy data points when the corresponding line-segment goes through the majority class areas.

Differently, MWMOTE [34] assigns higher weights for minority class samples that are nearer to the majority class areas. Additionally, in order to fit abnormal data distributions, MWMOTE does not select k nearest neighbours but same-cluster samples for data generation. However, due to the fact that the clustering algorithm only groups the minority class samples into clusters previously, by using pairwise distances, it does not tend to adaptively group some far minority class samples of the same minority class area together and some near minority class samples of different minority class areas apart.

On the other hand, as one of structure-preserving techniques, INOS [35] uses the main covariance structure of minority class for data generation. Besides, MDO [25] and AMDO [36] use the Mahalanobis distance from the minority class for data generation, while SWIM [37] uses the Mahalanbois distance from the majority class for data generation. Moreover, GDO [27] uses the Gaussian distribution for data generation. However, such structures do not take abnormal data distributions (like outliers, class overlapping or small disjuncts [31]) into consideration at all, so operating the data generation by means of preserving the corresponding structure may result in overly extended new data containing the noisy and overlapped data points.

3 Disjuncts-Robust Oversampling for Imbalance Problem

As seen in Fig. 1, our approach is inspired by the scene that uses multi horizontal searchlights to discourage or track trespassers in real life, by means of illuminating the restricted areas temporarily. So, how does those searchlights illuminate or scan the restricted areas in real life? Intuitively, a light cone is first started from the launch point, then passes through the restricted area, last is stopped by the surface of barriers; finally, a series of light cones are used to illuminate or scan the whole restricted areas. So, how to transfer this scene into the data filling for minority oversampling? Similarly, for each minority class point, we compute a light-cone structure that is first started from the inner minority class area, then passes through the seed minority class sample and it's nearby area, last is stopped by the majority class area.

In the following, we first give the definition of light-cone structure in $\mathbb{R}^n$. Second, we compute the light-cone structures. Finally, we fill those structures with synthetic data points.

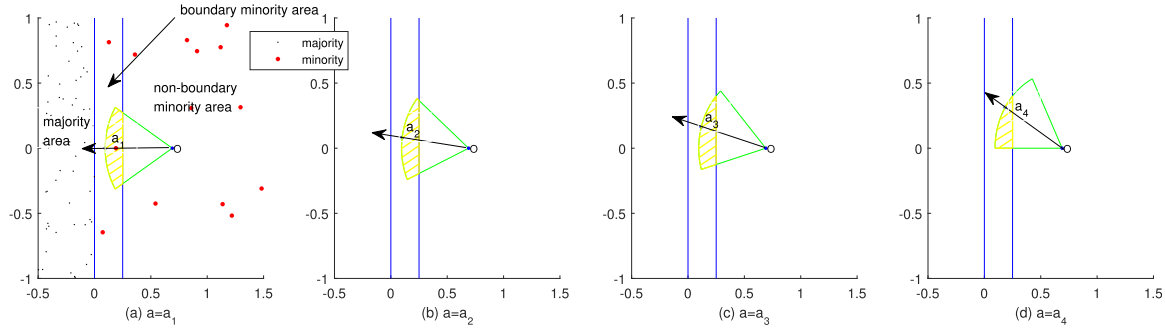


Fig. 2. Different intersection areas between the boundary minority class area and the light-cone structure area under varying a.

3.1 Definition of Light-Cone Structure

Intuitively, in real life, the light cone includes four components as one launch point, one illuminated direction, one cone angle and one distance of propagation. Except for the cone angle, another three components can be easily defined in $\mathbb{R}^n$. Thus for the cone angle, as seen in Fig. S1, which can be found on the Computer Society Digital Library at http://doi.ieeecomputersociety.org/10.1109/TKDE.2022.3161291, we replace the cone angle with the inner product of a unit borderline vector and a unit base vector. Thus, in $\mathbb{R}^n$, the light-cone structure also includes four corresponding components as one vertex, one base unit vector, one user-defined scalar parameter, and one radius.

Suppose the vertex is at the origin, we define the light-cone structure as

$$\mathbb{S} = \left\{ \boldsymbol{x} \middle| \left\langle \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_2}, \boldsymbol{a} \right\rangle \geq \rho \ \& \ \|\boldsymbol{x}\|_2 \leq r \right\} \bigcup \left\{ \boldsymbol{0} \right\}, \tag{1}$$

where $\left\langle , \right\rangle$ denotes the inner product operation of two vectors such as $\left\langle \boldsymbol{a}, \boldsymbol{b} \right\rangle = \boldsymbol{a} \bullet \boldsymbol{b}$, $\boldsymbol{a}$ is the base unit vector that $\|\boldsymbol{a}\|_2 = 1$, and $\rho \in [0, 1]$ is the user-defined scalar parameter, r is the radius.

Then, we transfer this structure as the intersection of an euclidean ball $\mathbb{B}$ and an unknown structure $\mathbb{C}$

$$\mathbb{C} = \left\{ \boldsymbol{x} \middle| \left\langle \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_2}, \boldsymbol{a} \right\rangle \geq \rho \right\} \bigcup \left\{ \boldsymbol{0} \right\} \tag{2}$$

$$\mathbb{B} = \left\{ \boldsymbol{x} \middle| \ \|\boldsymbol{x}\|_2 \leq r \right\} \tag{3}$$

$$\mathbb{S} = \mathbb{C} \bigcap \mathbb{B}. \tag{4}$$

To this end, we discuss the geometry characteristics of $\mathbb{C}$; in brief, whether $\mathbb{C}$ is a cone or a convex cone in $\mathbb{R}^n$.

Definition 3.1 (page 25 in [38]). *A set $\mathbb{C}$ is a cone, only for every $\boldsymbol{x} \in \mathbb{C}$ and $\theta \geq 0$, have $\theta\boldsymbol{x} \in \mathbb{C}$.*

Theorem 3.1. *Given a set $\mathbb{C} = \left\{ \boldsymbol{x} \middle| \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_2} \bullet \boldsymbol{a} \geq \rho \right\} \bigcup \left\{ \boldsymbol{0} \right\}$ where $\rho \in [-1, 1]$ is a scalar and $\boldsymbol{a}$ is a referred unit vector, the set $\mathbb{C}$ is a cone.*

Proof. Because, when $\boldsymbol{x} = \boldsymbol{0}$, $\theta\boldsymbol{x} = \boldsymbol{0} \in \mathbb{C}$. Thus, in the following, we suppose $\boldsymbol{x} \neq \boldsymbol{0}$.

Since $\boldsymbol{x} \in \mathbb{C}$, we have $\frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_2} \bullet \boldsymbol{a} \geq \rho$.

We consider the following two cases:

Case 1: $\theta = 0$: Have $\theta\boldsymbol{x} = \boldsymbol{0} \in \mathbb{C}$.

Case 2: $\theta > 0$, compute

$$\frac{\theta\boldsymbol{x}}{\|\theta\boldsymbol{x}\|_2} \bullet \boldsymbol{a} = \frac{\theta\boldsymbol{x}}{\theta\|\boldsymbol{x}\|_2} \bullet \boldsymbol{a} = \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_2} \bullet \boldsymbol{a} \geq \rho, \tag{5}$$

have $\theta\boldsymbol{x} \in \mathbb{C}$.

Hence, $\theta\boldsymbol{x} \in \mathbb{C}$ in both cases, $\mathbb{C}$ is a cone $\rho \in [-1, 1]$. □

Definition 3.2 (page 25 in [38]). *A set $\mathbb{C}$ is a convex cone if it is convex and a cone, which means that for any $\boldsymbol{x_1}, \boldsymbol{x_2} \in \mathbb{C}$ and $\theta_1, \theta_2 \geq 0$, we have*

$$\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2} \in \mathbb{C}. \tag{6}$$

Theorem 3.2. *Given a set $\mathbb{C} = \left\{ \boldsymbol{x} \middle| \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_2} \bullet \boldsymbol{a} \geq \rho \right\} \bigcup \left\{ \boldsymbol{0} \right\}$ where $\rho \in [0, 1]$ is a scalar and $\boldsymbol{a}$ is a referred unit vector, the set $\mathbb{C}$ is a convex cone.*

Proof. Because, when $\boldsymbol{x_1} = \boldsymbol{0}$ or $\boldsymbol{x_2} = \boldsymbol{0}$, $\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2} \in \mathbb{C}$. Thus, in the following, we suppose $\boldsymbol{x_1} \neq \boldsymbol{0}$ and $\boldsymbol{x_2} \neq \boldsymbol{0}$.

We first make $f(\boldsymbol{x}) = \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_2} \bullet \boldsymbol{a}$. Since $\boldsymbol{x_1}, \boldsymbol{x_2} \in \mathbb{C}$, we have $f(\boldsymbol{x_1}) = \frac{\boldsymbol{x_1}}{\|\boldsymbol{x_1}\|_2} \bullet \boldsymbol{a} \geq \rho$ and $f(\boldsymbol{x_2}) = \frac{\boldsymbol{x_2}}{\|\boldsymbol{x_2}\|_2} \bullet \boldsymbol{a} \geq \rho$.

We consider the following two cases:

Case 1: $\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2} = \boldsymbol{0} \in \mathbb{C}$.

Case 2: $\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2} \neq \boldsymbol{0}$, compute

$$\begin{aligned} f(\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2}) &= \frac{\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2}}{\|\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2}\|_2} \bullet \boldsymbol{a} \\ &= \frac{\theta_1\|\boldsymbol{x_1}\|_2\frac{\boldsymbol{x_1}}{\|\boldsymbol{x_1}\|_2} + \theta_2\|\boldsymbol{x_2}\|_2\frac{\boldsymbol{x_2}}{\|\boldsymbol{x_2}\|_2}}{\|\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2}\|_2} \bullet \boldsymbol{a} \\ &= \frac{\theta_1\|\boldsymbol{x_1}\|_2\frac{\boldsymbol{x_1}}{\|\boldsymbol{x_1}\|_2} \bullet \boldsymbol{a} + \theta_2\|\boldsymbol{x_2}\|_2\frac{\boldsymbol{x_2}}{\|\boldsymbol{x_2}\|_2} \bullet \boldsymbol{a}}{\|\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2}\|_2} \\ &= \frac{\theta_1\|\boldsymbol{x_1}\|_2 f(\boldsymbol{x_1}) + \theta_2\|\boldsymbol{x_2}\|_2 f(\boldsymbol{x_2})}{\|\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2}\|_2} \\ &\geq \frac{\theta_1\|\boldsymbol{x_1}\|_2 + \theta_2\|\boldsymbol{x_2}\|_2}{\|\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2}\|_2} \times \rho \\ &\geq \rho, \end{aligned} \tag{7}$$

have $\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2} \in \mathbb{C}$. Noting: the last step of proof is draw from the Triangle Inequality for norm ($\|\boldsymbol{x} + \boldsymbol{y}\|_2 \leq \|\boldsymbol{x}\|_2 + \|\boldsymbol{y}\|_2$).

Hence, $\theta_1\boldsymbol{x_1} + \theta_2\boldsymbol{x_2} \in \mathbb{C}$ in both cases; $\mathbb{C}$ is a convex cone when $\rho \in [0, 1]$. □

In general, if $\rho \in [-1, 1]$, the light-cone structure is the intersection of an euclidean ball and a cone; if $\rho \in [0, 1]$, the

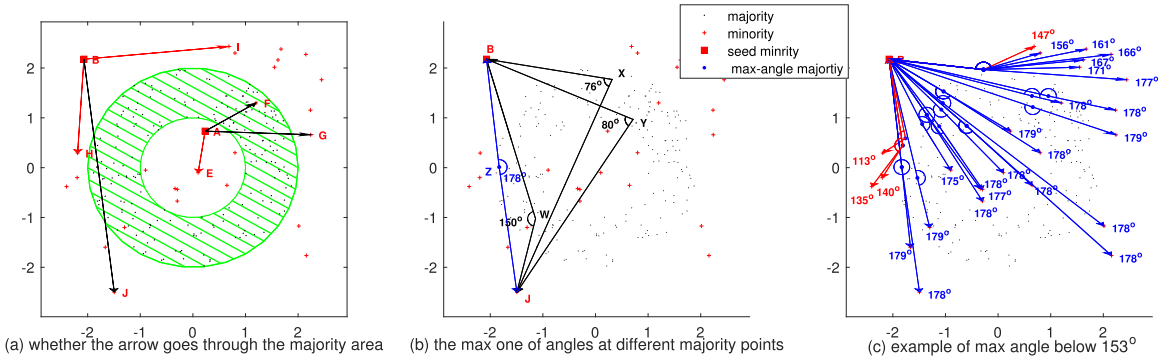


Fig. 3. The direct-interlinked relationship between a pair of minority class points. (a) whether the arrow goes through the majority class areas, (b) the max one of the angles at different majority points, (c) example of max angle below 153^o. In (a), the green slash denotes the majority class area, the red arrow denotes the line-segment does not go through the majority class area, the black arrow denotes the line-segment goes through the majority class area. In (b), the blue denotes the max one of the angles at different majority points. In (c), the red denotes the max angle of two minority points is smaller than the threshold 153^o, the blue denotes the max angle is larger than the threshold 153^o.

light-cone structure is the intersection of an euclidean ball and a convex cone.

3.2 Computation of Light-Cone Structure

3.2.1 Base Unit Vector

As seen in Fig. 2, we split the data space into the majority class area, boundary minority class area and non-boundary minority class area. As the base unit vector $\boldsymbol{a}$ changes from $\boldsymbol{a}_1$ to $\boldsymbol{a}_4$, the corresponding intersection area decreases. Thus, in order to cover more boundary area of minority class, we wish the base unit vector $\boldsymbol{a}$ that nearly equals to $\boldsymbol{a}_1$ in Fig. 2a.

In detail, for the seed minority class sample $\boldsymbol{x}$, we compute it's k nearest majority neighbours

$$S_{knn} = \{z_1, z_2, \ldots, z_k\}, \tag{8}$$

where z_k is the k-th nearest majority class sample. Then we compute the center

$$\bar{\boldsymbol{z}} = \frac{1}{k}\sum_{i=1}^{k} \boldsymbol{z}_i. \tag{9}$$

Last, we obtain the base unit vector $\boldsymbol{a}$

$$\boldsymbol{c} = \frac{\boldsymbol{x} - \bar{\boldsymbol{z}}}{\|\boldsymbol{x} - \bar{\boldsymbol{z}}\|_2} \tag{10}$$

$$\boldsymbol{a} = -c. \tag{11}$$

3.2.2 Vertex

Computating the vertex aims at launching the light-cone structure from it's corresponding inner minority class area by means of selecting other minority class samples that are of the same minority class area first. We then demonstrate how this selection can be used to attain a good location to the vertex that roots into the deeper minority class area than the seed minority class sample. Thus, in the first step, for the seed minority class sample, we select a group of other minority class samples that are of the same minority class area as it. As seen in Fig. 3a, two minority class points are of the same minority class area when their line-segment does not go through the majority class area. For example, for a pair of minority class point A and E, their line-segment does not go through the majority class area so they are of the same minority class area. In this study, we also call it the direct-interlinked relationship due to the line-segment directly interlinked by a pair of points. Howerve, directly judging whether the line-segment goes through the majority class areas is difficult while the majority class areas are unknown that only majority class points are known. Thus, to solve this problem, as seen in Fig. 3b, given a majority point X, we first compute the angle $\angle BXJ$ for a pair of minority class points B and J. Then, we go through all the majority points to find the maximum angle, for example $\angle BZJ = 178^o$ in this case. Last, we compare it with a user-defined angle, for example 153^o in Fig. 3c. Smaller angle than the user-defined one means two minority points are direct-interlinked, otherwise means not. For example, the maximum angle $\angle BZJ = 178^o$ is larger than the user-defined angle 153^o that minority class point B and J are not direct-interlinked. Thus, as seen in Fig. 3c, four minority points are direct-interlinked to the seed minority point B. For convenience, we do not directly compute the angle between three points, but compute their corresponding inner product in $\mathbb{R}^n$. For example, for $\angle BZJ = 178^o$, it's corresponding inner product is $cos(178^o)$.

In detail, for any two minority class samples $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$, we compute the inner product of two unit vectors

$$D_k(\boldsymbol{x}_i, \boldsymbol{x}_j, z_k) = \left\langle \frac{\boldsymbol{x}_i - z_k}{\|\boldsymbol{x}_i - z_k\|_2}, \frac{\boldsymbol{x}_j - z_k}{\|\boldsymbol{x}_j - z_k\|_2} \right\rangle, \tag{12}$$

where z_k is the k-th sample in the majority set. When $\|\boldsymbol{x}_i - z_k\|_2 = 0$ or $\|\boldsymbol{x}_j - z_k\|_2 = 0$, we set $D(\boldsymbol{x}_i, \boldsymbol{x}_j, z_k) = 0$. Then we pick the minimum inner product

$$M(\boldsymbol{x}_i, \boldsymbol{x}_j) = \min_{1 \le k \le |S_{maj}|} D_k(\boldsymbol{x}_i, \boldsymbol{x}_j, z_k), \tag{13}$$

where $|S_{maj}|$ is number of majority class samples. Finally, we obtain the relationship between $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$

$$I(\boldsymbol{x}_i, \boldsymbol{x}_j) = \begin{cases} 1, & if\ M(\boldsymbol{x}_i, \boldsymbol{x}_j) \ge \delta \\ 0, & else \end{cases}, \tag{14}$$

where δ is a user-defined threshold parameter ranging in [-1, 1]. $I(\boldsymbol{x}_i, \boldsymbol{x}_j) = 1$ denotes two minority direct-interlinked; otherwise, $I(\boldsymbol{x}_i, \boldsymbol{x}_j) = 0$ denotes not direct-interlinked.

In the second step, we use their direct-interlinked minority class samples to compute the vertex. For a seed minority point A, as seen in Fig. 4a, point B is the center of it's K

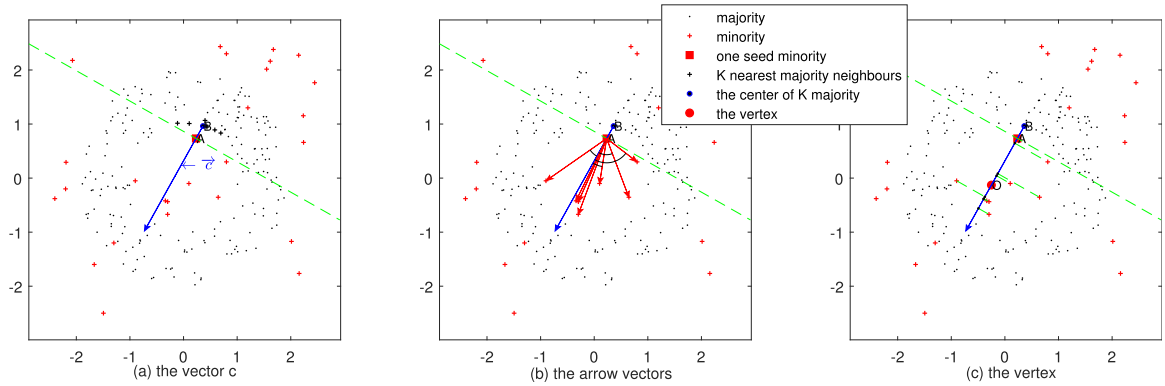


Fig. 4. Vertex computation. (a) the vector c, (b) the arrow vectors, (c) the vertex. Where the blue arrow denotes the vector c, green slashes denote the lines that vertical to c; red arrows are pointed to the direct-interlinked minority class points from the seed minority point.

majority neighbours, $\boldsymbol{c}$ is the inverse of it's base unit vector; as seen in Fig. 4b, we draw the arrow vectors from point A to it's direct-interlinked points; as seen in Fig. 4c, we project those arrow vectors on $\boldsymbol{c}$, then we use the mean point of positive projections as the vertex (point O).

In detail, for the seed minority class sample $\boldsymbol{x}$, we first obtain it's direct-interlinked minority class samples as $\{\boldsymbol{x}_1, \boldsymbol{x}_2, .., \boldsymbol{x}_i, .., \boldsymbol{x}_m\}$ that satisfying $I(\boldsymbol{x}, \boldsymbol{x}_i) = 1$, where $\boldsymbol{x}_i$ is i-th sample and m is the number of it's direct-interlinked minority class samples. Then, we obtain the arrow vectors

$$\{\boldsymbol{d}_1, \boldsymbol{d}_2, .., \boldsymbol{d}_i, .., \boldsymbol{d}_m\}, \tag{15}$$

where $\boldsymbol{d}_i$ is computed as

$$\boldsymbol{d}_i = \boldsymbol{x}_i - \boldsymbol{x}. \tag{16}$$

Then, we compute their projections on $\boldsymbol{c}$

$$\{p_1, p_2, .., p_i, .., p_m\}, \tag{17}$$

where p_i is computed as

$$p_i = \left\langle \boldsymbol{d}_i, \boldsymbol{c} \right\rangle. \tag{18}$$

Next, we compute the mean of positive projections

$$\bar{p} = \frac{1}{\sum_{i=1}^{m} J(p_i)} \sum_{i=1}^{m} J(p_i) \times p_i, \tag{19}$$

where $J(p_i)$ is the indicator function

$$J(p_i) = \begin{cases} 1, & if\ p_i > 0 \\ 0, & else \end{cases}. \tag{20}$$

Finally, we compute the vertex as

$$v = \bar{p} \times \boldsymbol{c} + \boldsymbol{x}. \tag{21}$$

3.2.3 Radius

As seen in Fig. 5a, for the seed point A, we first compute it's illuminated majority class samples; then, we select it's nearest majority class point C; last, we roughly compute the radius as the mean of OC and OA.

In detail, under the user-defined scalar parameter ρ, we first find illuminated majority class samples that satisfying

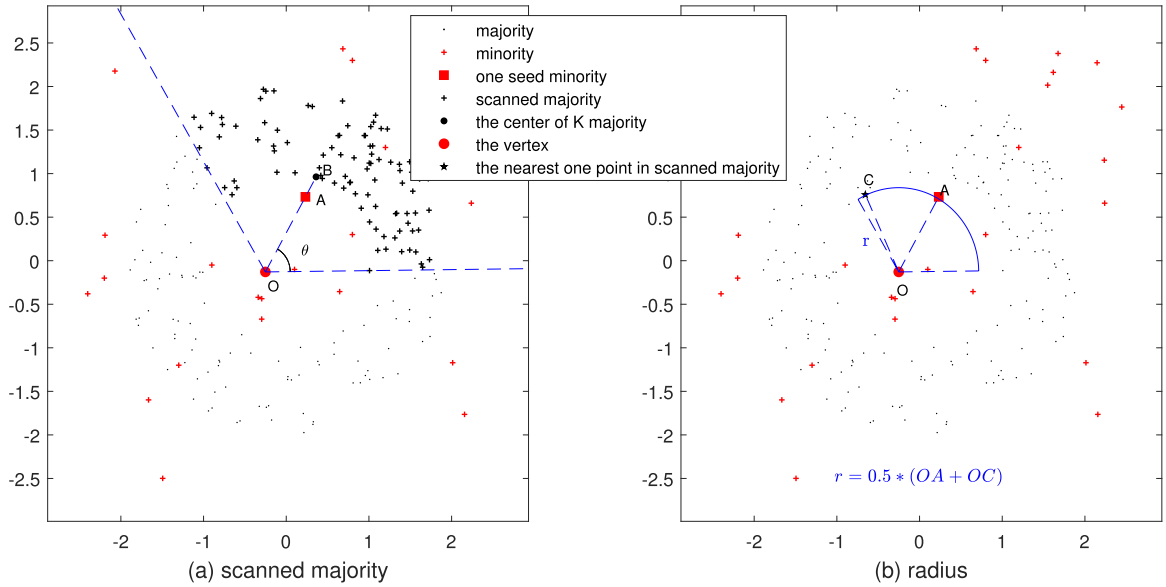


Fig. 5. Radius computation. (a) illuminated majority, (b) radius.

$$\left\langle \frac{\boldsymbol{z}_k - \boldsymbol{v}}{\|\boldsymbol{z}_k - \boldsymbol{v}\|_2}, \boldsymbol{c} \right\rangle \geq \rho, \tag{22}$$

where $\boldsymbol{z}_k$ is the k-th sample in the majority set, $\boldsymbol{v}$ is the vertex of light-cone structure. Next, we find it's nearest majority class sample as $\boldsymbol{g}$. Last, we compute the radius

$$r = min(L(\boldsymbol{v}, \boldsymbol{x}), L(\boldsymbol{v}, \boldsymbol{g})) + 0.5(L(\boldsymbol{v}, \boldsymbol{g}) - L(\boldsymbol{v}, \boldsymbol{x})), \tag{23}$$

where $min(,)$ means the minimum of two values, $L(,)$ is the euclidean distance that $L(v_1, v_2) = \|v_1 - v_2\|_2$. For convenience, we call $L(\boldsymbol{v}, \boldsymbol{g})$ as the length of g, $L(\boldsymbol{v}, \boldsymbol{x})$ as the length of x. Thus, if $L(\boldsymbol{v}, \boldsymbol{g})$ is larger than $L(\boldsymbol{v}, \boldsymbol{x})$, the radius r is the mean of two lengths; otherwise, $\boldsymbol{x}$ maybe the overlapped minority class point that computing the radius r by subtracting the half gap of two lengths from g. Thus, the light-cone structure tends to cover nearly half local boundary area in non-overlapped cases and not cover the majority class area in overlapping cases.

3.3 Data Generation in the Light-Cone Structure

Many oversampling techniques [8], [39] use imbalance-ratio (IR) to measure the binary-class imbalance degree for data generation. Besides, L. Li *et al.* [40] introduced the entropy-based imbalance degree (EID) to measure the multi-class imbalance degree.

Algorithm 1. DROS

Input: $S_{maj} = \{z_1, z_2, \ldots, z_{|S_{maj}|}\}$: the training majority class samples

$S_{min} = \{x_1, x_2, \ldots, x_{|S_{min}|}\}$: the training minority class samples

ρ : one of components of light-cone structure in Eq. (1)

k: used to search k nearest majority class samples in Eq. (8)

δ: the threshold in Eq. (14)

g: the controlling parameter in Eq. (25)

Output: Synthetic samples S_{new}

/* Step 1: compute the relationship between pairs of minority class samples */

$I(,) = Relationships(S_{maj}, S_{min}, |S_{maj}|, |S_{min}|, \delta)$

/* Step 2: Compute the light-cone structure for each minority class sample */

$S_2 = Structures(S_{maj}, S_{min}, |S_{maj}|, |S_{min}|, \rho, k, I(,))$

/* Step 3: generate synthetic samples */

$S_{new} = DataGeneration(|S_{maj}|, |S_{min}|, S_2)$

return S_{new}

In this study, we iteratively pick out one random light-cone structure from existed ones to generate a data point for $|S_{maj}| - |S_{min}|$ times, where the topic of generating $|S_{maj}| - |S_{min}|$ synthetic samples was previously addressed in [25] and [27]. In detail, we randomly pick out a light-cone structure first; then, we generate one data point in it

$$new = \boldsymbol{v} + (\xi \times r) \times \overrightarrow{d}, \tag{24}$$

where $\overrightarrow{d}$ is a rand unit vector that satisfying $\left\langle \overrightarrow{d}, \boldsymbol{a} \right\rangle \geq \rho$.

And ξ is a random scalar value in [g,1]

$$\xi = g + rand() \times (1 - g), \tag{25}$$

where rand() is a random value in [0,1]; g is a user-defined threshold parameter ranging in [0,1] that is used to control how near to the boundaries the points will be placed.

And we iteratively repeat above process for $|S_{maj}| - |S_{min}|$ times.

Algorithm 2. $Relationships(S_{maj}, S_{min}, |S_{maj}|, |S_{min}|, \delta))$

Output: Relationship matrix $I(,)$
for i=1 to $|S_{min}|$-1 **do**
  for j=i+1 to $|S_{min}|$ **do**
    for k=1 to $|S_{maj}|$ **do**
      $D_k(\boldsymbol{x}_i, \boldsymbol{x}_j, \boldsymbol{z}_k) = \left\langle \frac{\boldsymbol{x}_i - \boldsymbol{z}_k}{\|\boldsymbol{x}_i - \boldsymbol{z}_k\|_2}, \frac{\boldsymbol{x}_j - \boldsymbol{z}_k}{\|\boldsymbol{x}_j - \boldsymbol{z}_k\|_2} \right\rangle$;
    end for
    $M(\boldsymbol{x}_i, \boldsymbol{x}_j) = \min_{1 \leq k \leq |S_{maj}|} D_k(\boldsymbol{x}_i, \boldsymbol{x}_j, \boldsymbol{z}_k)$;
    $I(\boldsymbol{x}_i, \boldsymbol{x}_j) = 0$;
    if $M(\boldsymbol{x}_i, \boldsymbol{x}_j) \geq \delta$ **then**
      $I(\boldsymbol{x}_i, \boldsymbol{x}_j) = 1$;
    end if
  end for
end for
return $I(,)$

Algorithm 3. $Structures(S_{maj}, S_{min}, |S_{maj}|, |S_{min}|, \rho, k, I(,))$

Output: the light-cone structure set S_2
for i=1 to $|S_{min}|$ **do**
  $S_{knn} = \{z_1, z_2, \ldots, z_k\} = knnserach(S_{maj}, x_i, ' K', k)$;
  $\bar{\boldsymbol{z}} = \frac{1}{k} \sum_{i=1}^{k} \boldsymbol{z_i}$;
  $\boldsymbol{c} = \frac{\boldsymbol{x_i} - \bar{\boldsymbol{z}}}{\|\boldsymbol{x_i} - \bar{\boldsymbol{z}}\|_2}$
  $\boldsymbol{a} = -\boldsymbol{c}$;
  for j=1 to $|S_{min}|$ **do**
    $J(p_j) = 0$;
    if $I(\boldsymbol{x}_i, \boldsymbol{x}_j) == 1$ **then**
      $\boldsymbol{d}_j = \boldsymbol{x}_j - \boldsymbol{x_i}$;
      $p_j = \left\langle \boldsymbol{d}_i, \boldsymbol{c} \right\rangle$
      if $p_j > 0$ **then**
        $J(p_j) = 1$;
      end if
    end if
  end for
  $\bar{p} = \frac{1}{\sum_{j=1}^{|S_{min}|} J(p_j)} \sum_{i=1}^{|S_{min}|} J(p_j) \times p_j$
  $\boldsymbol{v} = \bar{p} \times \boldsymbol{c} + \boldsymbol{x_i}$
  for k=1 to $|S_{maj}|$ **do**
    $I(\boldsymbol{z}_k) = 0$;
    if $\left\langle \frac{\boldsymbol{z}_k - \boldsymbol{v}}{\|\boldsymbol{z}_k - \boldsymbol{v}\|_2}, \boldsymbol{c} \right\rangle \geq \rho$ **then**
      $I(\boldsymbol{z}_k) = 1$;
      $L(\boldsymbol{v}, \boldsymbol{z}_k) = \|\boldsymbol{v} - \boldsymbol{z}_k\|_2$;
      Add $L(\boldsymbol{v}, \boldsymbol{z}_k)$ to set S_1
    end if
  end for
  $L(\boldsymbol{v}, \boldsymbol{g}) = min\ S_1$
  $r = min(L(\boldsymbol{v}, \boldsymbol{x_i}), L(\boldsymbol{v}, \boldsymbol{g})) + 0.5(L(\boldsymbol{v}, \boldsymbol{g}) - L(\boldsymbol{v}, \boldsymbol{x_i}))$;
  if $\sum_{i=1}^{|S_{min}|} J(p_i) == 0$ *or* $\boldsymbol{a} == \boldsymbol{0}$ *or* $r \leq 0$ *or* $\sum_{i=1}^{|S_{maj}|} I(\boldsymbol{z}_k) == 0$ **then**
    Considering $\boldsymbol{x_i}$ as the improper point for data generation;
  else
    Add $s = \{\boldsymbol{a}, \boldsymbol{v}, r, \rho\}$ to S_2;
  end if
end for
return S_2

Algorithm 4. $DataGeneration(|S_{maj}|, |S_{min}|, S_2)$

Output: Synthetic samples S_{new}
for i=1 to $|S_{maj}| - |S_{min}|$ **do**
$s = \{\boldsymbol{a}, \boldsymbol{v}, r, \rho\}$ = a randomly picked light-cone structure from S_2;
$\xi = g + rand() \times (1 - g)$;
$\overrightarrow{d}$ = a randomly generated unit vector that satisfying $\left\langle \overrightarrow{d}, \boldsymbol{a} \right\rangle \geq \rho$;
$new = v + (\xi \times r) \times \overrightarrow{d}$;
Add new to S_{new};
end for
return S_{new}

3.4 DROS Algorithm and it's Computational Complexity

As seen in Algorithm 1, we provide the description of DROS algorithm. The input includes one training majority class set S_{maj}, one training minority class set S_{min}. Besides, the input also includes four user-defined parameters of DROS algorithm as ρ that serves as one of components of light-cone structure in Eq. (1); k that used to search k nearest majority class samples in Eq. (8); δ that denotes the threshold in Eq. (14); g that denotes the lower limit parameter in Eq. (25). The output is the set S_{new} that includes the synthetic samples.

In the first step, as seen in Algorithm 2, we compute the direct-interlinked relationship between each pair of minority class samples, this counts for $|S_{min}| \times (|S_{min}| - 1)/2$ times. First, for each pair (x_i, x_j), we go through all majority class samples to compute their corresponding inner products that counts for $|S_{maj}|$ times. Then, we find the minimum inner product, this counts for $|S_{maj}|$ times; and we compare it with a user-defined parameter δ; if $M(\boldsymbol{x}_i, \boldsymbol{x}_j) \geq \delta$, we record $I(\boldsymbol{x}_i, \boldsymbol{x}_j) = 1$. The computational complexity of this step is $O(|S_{min}|^2 \times |S_{maj}| \times H)$, where H denotes the dimension of data.

In the second step, as seen in Algorithm 3, for each minority class sample x_i, we compute it's corresponding light-cone structure. First, for x_i, we use $knnserach(S_{maj}, x_i, ' K', k)$ to compute it's k nearest majority class samples from S_{maj} that counts for $|S_{min}| + |S_{min}| - 1 + \cdots + |S_{min}| - k$ times, then we compute the mean point, next we make it pointed to x_i and normalize it as the direction vector, last we serve the inverse of direction vector as the base unit vector. Second, for x_i, we search it's direct-interlinked samples that counts for $|S_{min}|$ times. For the direct-interlinked minority class sample x_j, we first make x_i pointed to x_j; then, we project the pointed arrow onto $\boldsymbol{c}$; last, we serve the mean location of positive projections as the vertex. Third, we first search the scanned majority class samples that counts for $|S_{maj}|$ times; for the majority class sample z_k to be scanned, we add it's length to set S_1; then, we find the minimum length in S_1 where g denotes the corresponding nearest majority class sample, this counts for $|S_{maj}|$ times; last we compute the radius. Since four components (the base unit vector, the vertex and the radius and the user-defined parameter ρ) of light-cone structure have been obtained, then we judge whether the light-cone structure is proper for data generation. where $\sum_{i=1}^{|S_{min}|} J(p_i) == 0$ means the seed minority class sample owns no other minority samples direct-interlinked, we consider it as the noisy data point whose light-cone structure is not proper for data generation; $\boldsymbol{a} == \boldsymbol{0}$ means the seed minority class sample overlaps with the mean center of it's k nearest majority class samples, we consider it's light-cone structure not proper for data generation; $r \leq 0$ means the light-cone structure does not exist, we consider it's light-cone structure not proper for data generation; $\sum_{i=1}^{|S_{maj}|} I(\boldsymbol{z}_k) == 0$ means no scanned majority class samples exists, we consider it's light-cone structure not proper for data generation. Otherwise, we add $s = \{\boldsymbol{a}, \boldsymbol{v}, r, \rho\}$ to the light-cone structure set S_2. The computational complexity of this step is $O(|S_{min}| \times (|S_{maj}| + |S_{min}|) \times H)$, where H denotes the dimension of data.

In the third step, as seen in Algorithm 4, we iteratively repeat to generate one data point in the random picked light-cone structure for $|S_{maj}| - |S_{min}|$ times. In each time, we randomly pick a light-cone structure from set S_2 first; then, we generate a random scalar value in $[g, 1]$, next we generate a random direction vector, last we use them to obtain the new synthetic sample that added to S_{new}. Notice that in the high dimension, randomly generating the condition-satisfied unit vector is difficult. To solve this problem, we generate a random unit vector first, then add it to $\boldsymbol{a}$, next normalize it to a unit vector again; we repeat this procedure for several times (nearly as N_1 times) until a condition-satisfied unit vector $\overrightarrow{d}$ generated. In experience, we uniformly generate N_2 condition-satisfied unit vectors first, then randomly pick up one for use. Thus, the computational complexity of this step is $O((|S_{maj}| - |S_{min}|) \times N_1 \times N_2 \times H)$, where H denotes the dimension of data. Thus, the total computational complexity of DROS algorithm is $O([|S_{maj}||S_{min}|^2 + (|S_{maj}| - |S_{min}|)N_1 N_2] \times H)$.

In sum, the number of used light-cone structures is the size of set S_2, that only can be obtained after Step 2 of the DROS algorithm. And for the non-convex minority class area, we just use multiple light-cone structures to illuminate or scan them. Of curse, multiple light-cone structures may produce overlap areas. To a certain extent, the user-defined parameter g in Eq. (25) can be also used to control how many overlapped areas are produced. Because the larger g means the smaller area of light-cone structure for use that would produce smaller overlap areas.

4 Experimental Results

4.1 Evaluation Metrics

Accuracy is generally served as one of evaluation metrics for classiricatoin tasks. However, it tends to be specifically designed for balanced datasets and is thus not appropriately applicable on imbalanced datasets. Thus, in this study, we select recall, f-measure and g-mean as the evaluation metrics

$$\begin{aligned} precision &= \frac{TP}{TP + FP} \\ recall &= \frac{TP}{TP + FN} \\ f - measure &= \frac{2 \times recall \times precision}{recall + precision} \\ g - mean &= \sqrt{\frac{TP \times TN}{(TP + FN) \times (TN + FP)}}, \end{aligned} \tag{26}$$

where TP, TN, FN and FP are the number of true positives, true negatives, false negatives and false positives.

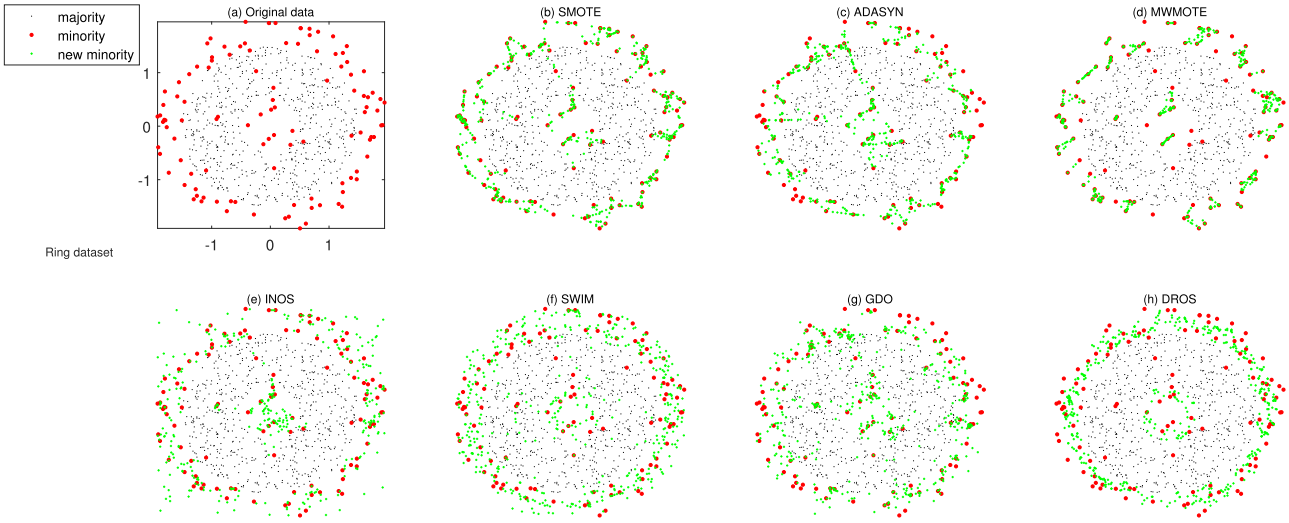


Fig. 6. Synthetic data on Ring dataset.

Besides, we also select auc as one of the evaluation metrics. In this study, auc means the area under an ROC curve, where ROC curve is a graph that shows the performance of a classification model at all possible threshold.

4.2 Experimental Setting

Our method can be applied only to instances with numerical dimensions. Thus, we carry on the compared experiments on four 2D emulational datasets and 31 real-world datasets. As seen in Fig. 6a, Figs. S2-S7a (in the supplementary material, available online), we plot the 2D emulational datasets respectively named as Ring, Curve1, Curve2, TwoBall1, TwoBall2, Linear1 and Linear2. As shown in Table 1, we provide the basic properties of real-world datasets collected from UCI repository [41] and [42], where this table uses 'Labels of selected classes' to denote the labels of selected minority class and majority class for

TABLE 1
Basic Properties of Real-World Datasets

dataset	Number of dimensions	Labels of selected classes	Number of instances	imbalance ratio
Survival $<$ 5yr	3	-	81:225	2.8
Biomed diseased	5	-	67:127	1.9
Cancer wpbc ret	33	-	47:151	3.2
Diabetes absent	8	-	268:500	1.9
Hepatitis normal	19	-	32:123	3.8
Housing MEDV $>$ 35	13	-	48:458	9.5
Spectf 0	44	-	95:254	2.7
Iris setosa	4	-	50:100	2.0
Abalone_{5-6}	8	5; 6	115:259	2.25
Abalone_{4-11}	8	4; 11	57:487	8.54
Ecoli_{4-2}	7	4; 2	35:77	2.20
Ecoli_{5-1}	7	5; 1	20:143	7.15
Glass_{7-2}	9	7; 2	29:76	2.62
Glass_{5-1}	9	5; 1	13:70	5.38
Pageblocks_{3-1}	10	3; 1	28:4913	175.46
Pageblocks_{5-2}	10	5; 2	115:329	2.86
$\text{WallFollowingRobotNav}_{2-3}$	24	2; 3	826:2097	2.54
$\text{WallFollowingRobotNav}_{4-1}$	24	4; 1	328:2205	6.72
Yeast_{5-3}	8	5; 3	51:244	4.78
Yeast_{9-4}	8	9; 4	20:163	8.15
Colon 1	1908	-	22:40	1.8
DMEAntiVirus	531	-	72:302	4.1944
Leukemia 1	3571	-	25:47	1.9
Metas 1	4919	-	46:99	2.2
ParkinsonsDC	754	-	564:192	2.9375
GLRCWL_{1-3}	698	1; 3	21:40	1.90
GLRCWL_{2-3}	698	2; 3	15:40	2.67
GLRCNBI_{1-3}	698	1; 3	21:40	1.90
GLRCNBI_{2-3}	698	2; 3	15:40	2.67
ARBT_{6-3}	8266	6; 3	12:162	13.50
ARBT_{5-4}	8266	5; 4	31:189	6.10

binary classification and uses '-' to denote the corresponding dataset is just for binary classification; uses 'Number of instances' to mean the number of minority class instances and majority class instances; use 'imbalance ratio' to mean the ratio between the number of majority and minority class instances. Before experience, those real-world datasets are pre-processed by the standardized z-scores. For each dataset, we use a stratified 5-fold cross validation, where the dataset is randomly divided into 5 folds. In the inner loop, each fold is used for testing once and the remaining 4 folds are used for training that leads to 5 runs. In the external loop, we repeat the above procedure for 10 times that leads to the results averaged over 50 runs.

We select Support Vector Machine (SVM, 'Gaussian' kernel, all settings can also refer to the default setting of Matlab function: fitcsvm(data, target)) and Neural Network (NN, one hidden layer (10 hidden nodes, 'tansig' activation function), one output layer ('softmax' activation function), all settings can also refer to the default setting of Matlab function: train(patternnet(hiddenSizes =10), data, target)) as the base classifiers.

We select other state-of-art oversampling methods including SMOTE [30], ADASYN [33], MWMOTE [34], INOS [35], SWIM [37] and GDO [27] for comparison. We repeated all the compared algorithms and reconstruct their codes on Matlab 2017b, Windows 10, 64 bits, Core i9 CPU, RAM 32.0 GB. The euclidean distance is used to measure the distance between samples. For fair comparison, we first generate $|S_{maj}| - |S_{min}|$ new samples for each oversampling methods; then, we combine those new samples with the raw data to train the classifiers; last, we use the test data to evaluate their classification performance. Besides, we denote Ori as the method that trained a classifier with the raw data for comparison.

About the parameter setting, we set $\rho = 0.5$, $\delta = -0.7660$, $k = 7$ and $g = 1$ for our method. These values were tuned after empirical testing by fixing another three parameters and testing the target one with varied values. The empirical test similarly uses a stratified 5-fold cross validation for 10 times, that leads to the results averaged over 50 runs. First, for $\rho = 0.5$, as seen in Fig. S8, available online, different g-mean and auc results are plotted with varying ρ on several picked datasets. ρ values larger than 0.8 produce the decrease performance on g-mean; because, as ρ gradually increases to 1, it's light-cone structure gradually becomes a line-segment; thus we set $\rho = 0.5$. Second, for $\delta = -0.7660$ and $k = 7$, as seen in Figs. S9 and S10, available online, different g-mean and auc results are plotted with varying δ and k on several picked datasets. For δ and k, their g-mean results do not change much by different settings, thus we just set $\delta = -0.7660$ and $k = 7$. Third, for $g = 1$, as seen in Fig. S11, available online, different g-mean and auc results are plotted with varying g on several picked datasets. For some datasets, the larger g brings the better g-mean, thus we set $g = 1$.

For other compared methods, we set the parameters with default values in their original texts, respectively as $K = 5$ in SMOTE [30], $K = 5$ in ADASYN [33], $k_1 = 5$, $k_2 = 3$, $k_3 = |S_{min}|/2 = |S_{min}|/2$, $C_p = 3$, $C_f(th) = 2$ and $CMAX = 2$ in MWMOTE [34], $r = 0.7$ in INOS [35], $\alpha = 2$ in SWIM [37], and $K = 5$ and $\alpha = 1$ in GDO [27].

4.3 Visual Comparison in $\mathbb{R}^2$ Datasets

This section plots synthetic data of oversampling methods for visual comparison on 2D emulational datasets, respective named Ring, Curve1, Curve2, TwoBall1, TwoBall2, Linear1 and Linear2 in Fig. 6, Fig. S2-S7 (in the supplementary material, available online). Black points denote majority class samples, red points denote minority class samples and green crosses denote synthetic minority class samples. Besides, we add noisy data points to the Ring, Curve2, Linear2 and TwoBall2 datasets; we add overlapped points to the Ring, Linear1 and Linear2 datasets; we add small disjuncts to the Ring, Curve2, Linear2, TwoBall1 and TwoBall2 datasets.

As seen in Figs. 6, S4 and S5, available online, our method just generates synthetic samples along their borderlines. As seen in Figs. 6, S3-S5 and S7, available online, our method is of natural robustness to small disjuncts and is of good robustness to noisy data points. Besides, as seen in Figs. 6, S6 and S7, available online, our method is of good robustness to overlapped points.

As seen in Table S2, available online, we provide the performance evaluation for the synthetic datasets as well. our method obtains the good rank on the Ring, Curve1, Curve2, TwoBall2 and Linear2 datasets. These results indicate that our method is of good robustness to small disjuncts and noisy data points.

4.4 Evaluation on Real-World Data Sets

For the real-world datasets, Tables 2 and S2-S5, available online, summarize the average classification results of SVM classifier respectively in g-mean, precision, recall, f-measure and auc. Tables S6-S10, available online, summarize the average classification results of NN classifier respectively in precision, recall, f-measure, g-mean and auc. As shown in Table 2, each table cell includes the mean and standard deviation of 50 runs, and the technique's rank in a bracket; and in each row, the best rank is highlighted as bold. For a rough comparison, as shown in Figs. S12 and S13, available online, we count the best-rank times for each method over all datasets respectively for the SVM classifier and NN classifier.

4.4.1 Mean Rank

As seen in Tables 3 and S11, available online, we provide the mean of the ranks on all the datasets. As shown in Table 3, we compute the corresponding mean ranks for g-mean, precision, recall, f-measure and auc. In each row, the best mean rank is highlighted as bold. As seen in Table 3, except for precision, our method achieves the best mean ranks on all evaluation metrics. Because generating new samples near to the borderline tends to loss in precision and gain in recall.

4.4.2 Friedman Test and Posthoc Bonferroni-Dunn Test

As shown in Tables 3 and S11, available online, we use the Friedman test (alpha=0.05) to judge whether there is a significant difference between the compared groups. If the actual value is larger than the reference value, there is a significant difference between the compared groups so that we mark this actual value with a $reject$ in a bracket. These

TABLE 2
Average g-Mean With an SVM Classifier

Dataset	Ori	SMOTE	ADASYN	MWMOTE	INOS	SWIM	GDO	DROS
Survival <5yr	0.0366±0.1027(8)	0.5077±0.0829(4)	0.5027±0.0917(5)	0.5143±0.0865(3)	0.5175±0.0878(2)	0.5008±0.0903(6)	0.4977±0.0902(7)	**0.5224±0.0861(1)**
Biomed diseased	0.8396±0.0821(8)	0.8522±0.0849(6)	**0.8635±0.0703(1)**	0.8569±0.0744(4)	0.8485±0.0747(7)	0.8572±0.0769(3)	0.8544±0.0723(5)	0.8628±0.0743(2)
Cancer wpbc ret	0.4942±0.1495(8)	0.6550±0.1096(5)	0.6688±0.0953(3)	0.6606±0.1094(4)	0.6413±0.1247(6)	0.6800±0.0955(2)	0.6308±0.1135(7)	**0.6851±0.0985(1)**
Diabetes absent	0.6363±0.0444(8)	0.7019±0.0441(6)	0.7050±0.0388(2)	**0.7064±0.0401(1)**	0.6912±0.0389(7)	0.7037±0.0417(4)	0.7023±0.0399(5)	0.7046±0.0389(3)
Hepatitis normal	0.6087±0.2021(8)	0.7051±0.1385(5)	0.6987±0.1295(6)	0.7213±0.1422(3)	0.6877±0.1576(7)	**0.7545±0.1302(1)**	0.7200±0.1317(4)	0.7446±0.1179(2)
Housing MEDV>35	0.6917±0.1049(8)	0.8710±0.0753(2)	0.8707±0.0703(3)	0.8553±0.0785(7)	0.8621±0.0744(6)	0.8647±0.0504(5)	0.8691±0.0587(4)	**0.8790±0.0526(1)**
Spectf 0	0.7117±0.0636(8)	0.7828±0.0526(2)	**0.7848±0.0585(1)**	0.7551±0.0556(7)	0.7634±0.0681(6)	0.7659±0.0583(5)	0.7674±0.0551(4)	0.7765±0.0564(3)
Iris setosa	0.9897±0.0207(5.5)	0.9897±0.0207(5.5)	0.9897±0.0207(5.5)	0.9897±0.0207(5.5)	0.9897±0.0207(5.5)	0.9908±0.0199(2)	0.9897±0.0207(5.5)	**1.0000±0.0000(1)**
$Abalone_{5-6}$	0.5003±0.0886(8)	0.7222±0.0644(3)	0.7193±0.0588(6)	0.7100±0.0653(7)	0.7205±0.0566(5)	**0.7367±0.0534(1)**	0.7269±0.0602(2)	0.7218±0.0600(4)
$Abalone_{4-11}$	0.9684±0.0430(8)	0.9795±0.0288(2)	0.9774±0.0282(5)	0.9751±0.0389(6)	**0.9796±0.0291(1)**	0.9776±0.0261(4)	0.9751±0.0280(7)	0.9787±0.0292(3)
$Ecoli_{4-2}$	0.6500±0.1230(8)	0.7359±0.1160(3)	**0.7409±0.1042(1)**	0.7226±0.1114(7)	0.7297±0.1057(6)	0.7332±0.0877(5)	0.7364±0.1157(2)	0.7336±0.1015(4)
$Ecoli_{5-1}$	0.9493±0.0647(3.5)	0.9493±0.0647(3.5)	0.9467±0.0654(5.5)	0.9467±0.0654(5.5)	0.9664±0.0572(2)	0.9346±0.0755(8)	0.9347±0.0810(7)	**0.9757±0.0488(1)**
$Glass_{7-2}$	0.9062±0.0950(8)	0.9080±0.0926(7)	0.9223±0.0755(4)	0.9086±0.0931(6)	0.9166±0.0766(5)	0.9314±0.0617(2)	0.9239±0.0813(3)	**0.9404±0.0671(1)**
$Glass_{5-1}$	0.9449±0.1668(5.5)	0.9449±0.1668(5.5)	0.9449±0.1668(5.5)	0.9449±0.1668(5.5)	0.9366±0.2091(8)	0.9855±0.0489(2)	0.9758±0.0802(3)	**0.9941±0.0414(1)**
$Pageblocks_{3-1}$	0.6665±0.1677(8)	0.9519±0.0852(5)	0.9345±0.0909(6)	0.9129±0.0916(7)	0.9934±0.0320(2)	0.9701±0.0736(4)	**0.9940±0.0148(1)**	0.9924±0.0319(3)
$Pageblocks_{5-2}$	0.9240±0.0335(7)	0.9364±0.0303(6)	0.9595±0.0198(2)	0.9370±0.0352(5)	0.9532±0.0227(3)	0.9029±0.0365(8)	**0.9641±0.0146(1)**	0.9400±0.0315(4)
$WallFollowingRobotNav_{2-3}$	0.7580±0.0250(8)	0.8885±0.0127(2)	0.8739±0.0142(4)	**0.8941±0.0154(1)**	0.8589±0.0209(6)	0.8324±0.0215(7)	0.8648±0.0151(5)	0.8761±0.0149(3)
$WallFollowingRobotNav_{4-1}$	0.7780±0.0418(8)	0.9128±0.0201(4)	0.8887±0.0236(6)	0.9099±0.0238(5)	0.9131±0.0215(3)	0.9236±0.0148(2)	0.8827±0.0228(7)	**0.9270±0.0166(1)**
$Yeast_{5-3}$	0.7023±0.0898(8)	0.8006±0.0893(6)	0.8236±0.0661(3)	0.8017±0.0671(5)	0.7942±0.0804(7)	**0.8305±0.0651(1)**	0.8178±0.0681(4)	0.8261±0.0732(2)
$Yeast_{9-4}$	0.8080±0.2306(8)	0.8880±0.1170(5)	0.8838±0.1171(6)	0.8938±0.1237(4)	0.9262±0.1078(2)	0.8109±0.1737(7)	0.9259±0.0914(3)	**0.9512±0.0581(1)**
Colon 1	0.6168±0.2192(6)	0.6168±0.2192(6)	0.6168±0.2192(6)	0.6168±0.2192(6)	0.6258±0.2042(3)	0.6825±0.1919(2)	0.6168±0.2192(6)	**0.7907±0.1279(1)**
DMEAntiVirus	0.9604±0.0350(6)	0.9604±0.0350(6)	0.9604±0.0350(6)	0.9604±0.0350(6)	0.9626±0.0354(3)	0.9604±0.0350(6)	0.9641±0.0343(2)	**0.9850±0.0172(1)**
Leukemia 1	0.8195±0.1443(5.5)	0.8195±0.1443(5.5)	0.8195±0.1443(5.5)	0.8195±0.1443(5.5)	0.8198±0.1427(3)	0.8076±0.1423(8)	0.8219±0.1446(2)	**0.9599±0.0477(1)**
Metas 1	0.2570±0.1689(6.5)	0.2570±0.1689(6.5)	0.2570±0.1689(6.5)	0.2570±0.1689(6.5)	0.2610±0.1630(3)	0.3670±0.1632(2)	0.2600±0.1714(4)	**0.5225±0.1095(1)**
ParkinsonsDC	0.7003±0.0794(7)	0.7003±0.0794(7)	0.7004±0.0793(5)	0.7003±0.0794(7)	0.7139±0.0668(4)	0.7199±0.0372(2)	0.7161±0.0735(3)	**0.7600±0.0427(1)**
$GLRCWL_{1-3}$	0.6249±0.1623(6)	0.6249±0.1623(6)	0.6249±0.1623(6)	0.6249±0.1623(6)	0.6262±0.1569(3)	0.7072±0.1510(2)	0.6249±0.1623(6)	**0.7552±0.1527(1)**
$GLRCWL_{2-3}$	0.3006±0.3242(6)	0.3006±0.3242(6)	0.3013±0.3248(4)	0.3006±0.3242(6)	0.2868±0.3211(8)	0.4695±0.3214(2)	0.3080±0.3327(3)	**0.6037±0.2233(1)**
$GLRCNBI_{1-3}$	0.7380±0.2106(5.5)	0.7380±0.2106(5.5)	0.7380±0.2106(5.5)	0.7380±0.2106(5.5)	0.7447±0.1884(3)	0.7580±0.1933(2)	0.7370±0.2141(8)	**0.8140±0.1377(1)**
$GLRCNBI_{2-3}$	0.3096±0.3126(5)	0.3096±0.3126(5)	0.3096±0.3126(5)	0.3096±0.3126(5)	0.2991±0.3149(8)	0.4389±0.3245(2)	0.3096±0.3126(5)	**0.5377±0.2722(1)**
$ARBT_{6-3}$	0.0000±0.0000(5)	0.0000±0.0000(5)	0.0000±0.0000(5)	0.0000±0.0000(5)	0.0000±0.0000(5)	0.0000±0.0000(5)	0.0000±0.0000(5)	**0.6501±0.3310(1)**
$ARBT_{5-4}$	0.0000±0.0000(5.5)	0.0000±0.0000(5.5)	0.0000±0.0000(5.5)	0.0000±0.0000(5.5)	0.0082±0.0577(2)	0.0000±0.0000(5.5)	0.0000±0.0000(5.5)	**0.9773±0.0386(1)**

TABLE 3
Mean Ranks on All Datasets With an SVM Classifier

Measurement	Actual value(Friedman test)	Ori	SMOTE	ADASYN	MWMOTE	INOS	SWIM	GDO	DROS
precision	31.16(reject)	**3.02**	4.03	5.10	4.18	3.87	5.32	5.53	4.95
recall	99.80(reject)	6.90†	5.26†	4.61†	5.39†	4.85†	3.10	4.08†	**1.81**
f-measure	55.74(reject)	6.31†	4.66†	4.56†	4.53†	4.27†	4.60†	5.03†	**2.03**
g-mean	84.27(reject)	6.89†	4.89†	4.53†	5.24†	4.56†	3.79†	4.39†	**1.71**
auc	41.20(reject)	6.02†	4.37†	4.60†	4.24†	4.21	5.34†	4.68†	**2.55**
the Friedman Test: F=14.07, (n=8-1,alpha=0.05)									
the Bonferroni-Dunn test: critical values=1.67, (k=8,alpha=0.05)									

TABLE 4
Wilcoxon Signed Rank test (Alpha=0.05) With an SVM Classifier

recall		auc	
Ours vs.	p-V	Ours vs.	p-V
SWIM	**0.0031**	INOS	**0.0237**

results show that there is a significant difference between the compared groups for all evaluation metrics.

Thus, we use the posthoc Bonferroni-Dunn test (alpha=0.05) to judge whether there is a significant difference between any two methods. In this study, we only compare our method with other ones. If the gap between two mean ranks is larger than the critical value (CD), there is a significant difference between these two methods. In other words, our method outperforms the compared one so that we mark a symbol † on the top right of mean rank of this compared method. As seen in Table 3, for f-measure and g-mean, our method outperforms all the other methods.

4.4.3 Wilcoxon Paired Signed-Rank Test

Moreover, as show in Tables 4 and S12, available online, we use the Wilcoxon paired signed-rank test (alpha=0.05) to judge whether there is a significant difference between any two methods. In this study, we only care the evaluation metrics that our method obtains the best mean rank and only compare it with no †-marked methods. If $p-value$ is below 0.05, there is a significant difference between two methods. In other words, our method outperforms the compared one, and we highlight this $p-value$ as bold.

As seen in Table S12, available online, for g-mean, our method outperforms the MWMOTE, INOS and GDO methods. But our method does not outperform the SMOTE and ADASYN methods. The reason is that, for some datasets, the average g-mean results between our method and the compared one are very close. And for some datasets, the compared one just obtains the better g-mean results.

In sum, our method obtains the best mean ranks of recall and g-mean on both classifiers, and the best mean rank of f-measure and auc on SVM. This implies the good balance of proposed method between truly classified minority class samples and wrongly classified majority class samples.

4.5 Running Time

Table 5 shows the time consuming of different oversampling methods. For some datasets, INOS is faster than our method, and for others, our method is faster than INOS. To Explain this discrepancy, as seen in Figs. S14-S17, available online, for each algorithm, we generate charts respectively plotting: Running time X number of dimensions, number of minority class samples, number of majority class samples, and difference (or ratio) between the majority and minority. In these charts, each point represents the running time of

TABLE 5
Comparison of Computation Time (Seconds) of Different Methods for Real-World Data Sets

Dataset	SMOTE	ADASYN	MWMOTE	INOS	SWIM	GDO	DROS
Survival <5yr	0.00	0.00	0.00	0.02	0.00	0.00	0.04
Biomed diseased	0.00	0.00	0.00	0.01	0.00	0.00	0.04
Cancer wpbc ret	0.00	0.00	0.00	0.02	0.00	0.01	0.07
Diabetes absent	0.00	0.00	0.01	0.03	0.00	0.01	0.72
Hepatitis normal	0.00	0.00	0.00	0.01	0.00	0.00	0.03
Housing MEDV >35	0.00	0.00	0.00	0.04	0.00	0.01	0.09
Spectf 0	0.00	0.00	0.00	0.04	0.00	0.01	0.58
Iris setosa	0.00	0.00	0.00	0.00	0.00	0.00	0.02
Abalone_{5-6}	0.00	0.00	0.00	0.01	0.00	0.00	0.12
Abalone_{4-11}	0.00	0.00	0.00	0.03	0.00	0.01	0.07
Ecoli_{4-2}	0.00	0.00	0.00	0.00	0.00	0.00	0.02
Ecoli_{5-1}	0.00	0.00	0.00	0.01	0.00	0.00	0.01
Glass_{7-2}	0.00	0.00	0.00	0.00	0.00	0.00	0.01
Glass_{5-1}	0.00	0.00	0.00	0.00	0.00	0.00	0.01
Pageblocks_{3-1}	0.00	0.01	0.02	2.30	0.00	0.14	0.44
Pageblocks_{5-2}	0.00	0.00	0.00	0.02	0.00	0.01	0.14
$\text{WallFollowingRobotNav}_{2-3}$	0.00	0.02	0.06	0.75	0.01	0.04	127.03
$\text{WallFollowingRobotNav}_{4-1}$	0.00	0.01	0.02	0.62	0.00	0.06	20.35
Yeast_{5-3}	0.00	0.00	0.00	0.02	0.00	0.01	0.04
Yeast_{9-4}	0.00	0.00	0.00	0.01	0.00	0.00	0.02
Colon 1	0.00	0.00	0.00	0.47	0.09	0.49	0.26
DMEAntiVirus	0.00	0.00	0.01	0.30	0.01	0.44	1.75
Leukemia 1	0.00	0.00	0.01	2.82	0.75	2.65	0.54
Metas 1	0.01	0.01	0.02	16.44	3.11	12.94	3.33
ParkinsonsDC	0.01	0.02	0.05	1.61	0.03	1.56	24.62
GLRCWL_{1-3}	0.00	0.00	0.00	0.06	0.01	0.07	0.10
GLRCWL_{2-3}	0.00	0.00	0.00	0.08	0.01	0.09	0.11
GLRCNBI_{1-3}	0.00	0.00	0.00	0.06	0.01	0.07	0.10
GLRCNBI_{2-3}	0.00	0.00	0.00	0.08	0.01	0.09	0.11
ARBT_{6-3}	0.01	0.01	0.01	152.07	20.91	136.99	5.09
ARBT_{5-4}	0.01	0.03	0.01	236.07	19.45	144.29	8.21

each algorithm as a function of all datasets. We provide different colors for different algorithms. In the x-coordinate, take the Fig. S14, available online, for example, it's first row of x-coordinate is the dimensions by an increasing order, the second row is the corresponding names of dataset. As seen in Fig. S15, available online, in general, our method costs more time as the number of minority class samples increases. As seen in Fig. S14, available online, INOS costs more time as the number of dimensions increases.

4.6 Limitation and Drawbacks

It has been shown that our method is advantageous when the goal is to deal with the binary classification problems, rather than dealing with the multi-class classification problems. Thus, for dealing with multi-class contexts, we may go through each minority class and treat all the remained as the majority class. However, this can be time-consuming for some datasets.

Our method specializes in addressing the small disjuncts problem, not for outliers or overlapping. Besides, our method has a high time complexity, which is not a desirable property for dealing with big datasets (with tens or hundreds of thousands of instances).

5 Conclusion

In this study, we present a novel oversampling method (DROS) to address the class-imbalance problem with the small disjuncts problem. The novel method tactfully treats the majority class areas as the barrier of buildings. Thus, in the first step, our method computes a series of light cones to illuminate the minority class areas, where each light-cone is first launched from the inner minority class area, then passes through the boundary minority class area, last is stopped by the majority class area. In the second step, our method fills these light cones with synthetic samples. Additionally, our findings suggest that a pair of minority class points is direct-interlinked when their line-segment does not go through the majority class areas.

Visualization results on emulational 2D datasets show the satisfied capability of our method to the small disjuncts problem. Classification results on real-world datasets show it's superior learning performance when compared with selected stat-of-the-art oversampling methods. In the future, we will attempt to improve it's robustness towards noisy and overlapped data points and to reduce it's time complexity for dealing with big datasets.

References

[1] A. D. Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784–3797, Aug. 2018.

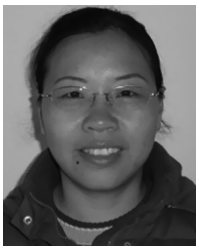
[2] H. Lin, G. Liu, J. Wu, Y. Zuo, X. Wan, and H. Li, "Fraud detection in dynamic interaction network," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 10, pp. 1936–1950, Oct. 2020.

[3] J. Jia, L. Zhai, W. Ren, L. Wang, and Y. Ren, "An effective imbalanced JPEG steganalysis scheme based on adaptive cost-sensitive feature learning," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 3, pp. 1038–1052, Mar. 2022.
[4] A. Tayal, T. F. Coleman, and Y. Li, "RankRC: Large-scale nonlinear rare class ranking," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 12, pp. 3347–3359, Dec. 2015.
[5] J. Hu, H. Yang, M. R. Lyu, I. King, and A. M.-C. So, "Online nonlinear AUC maximization for imbalanced data sets," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 4, pp. 882–895, Apr. 2018.
[6] C. Huang, C. C. Loy, and X. Tang, "Discriminative sparse neighbor approximation for imbalanced learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 5, pp. 1503–1513, May 2018.
[7] J. Mathew, C. K. Pang, M. Luo, and W. H. Leong, "Classification of imbalanced data by oversampling in kernel space of support vector machines," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 9, pp. 4065–4076, Sep. 2018.
[8] C.-T. Li *et al.*, "Minority oversampling in kernel adaptive subspaces for class imbalanced datasets," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 5, pp. 950–962, May 2018.
[9] X. Zhang, D. Ma, L. Gan, S. Jiang, and G. Agam, "CGMOS: Certainty guided minority oversampling," in *Proc. 25th ACM Int. Conf. Inf. Knowl. Manage.*, 2016, pp. 1623–1631.
[10] B. Cao, Y. Liu, C. Hou, J. Fan, B. Zheng, and J. Yin, "Expediting the accuracy-improving process of SVMs for class imbalance learning," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 11, pp. 3550–3567, Nov. 2021.
[11] M. Z. Jan, J. C. Munoz, and M. A. Ali, "A novel method for creating an optimized ensemble classifier by introducing cluster size reduction and diversity," *IEEE Trans. Knowl. Data Eng.*, early access, Sep. 22, 2020, doi: 10.1109/TKDE.2020.3025173.
[12] S. Ren *et al.*, "Selection-based resampling ensemble algorithm for nonstationary imbalanced stream data learning," *Knowl.-Based Syst.*, vol. 163, pp. 705–722, Jan. 2019.
[13] C. L. Castro and A. P. Braga, "Novel cost-sensitive approach to improve the multilayer perceptron performance on imbalanced data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 24, no. 6, pp. 888–899, Jun. 2013.
[14] W. Zong, G.-B. Huang, and Y. Chen, "Weighted extreme learning machine for imbalance learning," *Neurocomputing*, vol. 101, pp. 229–242, 2013.
[15] C. Zhang, K. C. Tan, H. Li, and G. S. Hong, "A cost-sensitive deep belief network for imbalanced classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 1, pp. 109–122, Jan. 2019.
[16] F. Wu, X.-Y. Jing, S. Shan, and W. Z. J.-Y. Yang, "Multiset feature learning for highly imbalanced data classification," in *Proc. 31st AAAI Conf. Artif. Intell.*, 2017, pp. 1583–1589.
[17] P. Lim, C. K. Goh, and K. C. Tan, "Evolutionary cluster-based synthetic oversampling ensemble (ECO-ensemble) for imbalance learning," *IEEE Trans. Cybern.*, vol. 47, no. 9, pp. 2850–2861, Sep. 2017.
[18] K. Yang *et al.*, "Hybrid classifier ensemble for imbalanced data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 4, pp. 1387–1400, Apr. 2020.
[19] S. Datta, S. Nag, and S. Das, "Boosting with lexicographic programming: Addressing class imbalance without cost tuning," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 5, pp. 883–897, May 2020.
[20] M. Bader-El-Den , E. Teitei, and T. Perry, "Biased random forest for dealing with the class imbalance problem," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 7, pp. 2163–2172, Jul. 2019.
[21] R. Razavi-Far , M. Farajzadeh-Zanajni , B. Wang, M. Saif, and S. Chakrabarti, "Imputation-based ensemble techniques for class imbalance learning," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 5, pp. 1988–2001, May 2021.
[22] A. Manukyan and E. Ceyhan, "Classification of imbalanced data with a geometric digraph family," *J. Mach. Learn. Res.*, vol. 17, pp. 1–40, Jan. 2016.
[23] Y. Sun, K. Tang, L. L. Minku, S. Wang, and X. Yao, "Online ensemble learning of data streams with gradually evolved classes," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 6, pp. 1532–1545, Jun. 2016.
[24] Q. Kang, X. Chen, S. Li, and M. Zhou, "A noise-filtered undersampling scheme for imbalanced classification," *IEEE Trans. Cybern.*, vol. 47, no. 12, pp. 4263–4274, Dec. 2017.
[25] L. Abdi and S. Hashemi, "To combat multi-class imbalanced problems by means of over-sampling techniques," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 1, pp. 238–251, Jan. 2016.
[26] A. Pourhabib, B. K. Mallick, and Y. Ding, "Absent data generating classifier for imbalanced class sizes," *J. Mach. Learn. Res.*, vol. 16, pp. 2695–2724, Jan. 2015.
[27] Y. Xie, M. Qiu, H. Zhang, L. Peng, and Z. Chen, "Gaussian distribution based oversampling for imbalanced data classification," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 2, pp. 667–679, Feb. 2022.
[28] W. W. Y. Ng, J. Hu, D. S. Yeung, S. Yin, and F. Roli, "Diversified sensitivity-based undersampling for imbalance classification problems," *IEEE Trans. Cybern.*, vol. 45, no. 11, pp. 2402–2412, Nov. 2015.
[29] W.-C. Lin, C.-F. Tsai, Y.-H. Hu, and J.-S. Jhang, "Clustering-based undersampling in class-imbalanced data," *Inf. Sci.*, vol. 409/410, pp. 17–26, May 2017.
[30] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
[31] C. Seiffert, T. M. Khoshgoftaar, J. V. Hulse, and A. Folleco, "An empirical study of the classification performance of learners on imbalanced and noisy software quality data," *Inf. Sci.*, vol. 259, pp. 571–595, 2014.
[32] H. Han, W. Wang, and B. Mao, "Borderline-SMOTE: A new oversampling method in imbalanced data sets learning," in *Proc. Int. Conf. Intell. Comput.*, 2005, pp. 878–887.
[33] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, 2008, pp. 1322–1328.
[34] S. Barua, M. M. Islam, X. Yao, and K. Murase, "MWMOTE—Majority weighted minority oversampling technique for imbalanced data set learning," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 2, pp. 405–425, Feb. 2014.
[35] H. Cao, X.-L. Li, D. Y.-K. Woon, and S.-K. Ng, "Integrated oversampling for imbalanced time series classification," *IEEE Trans. Knowl. Data Eng.*, vol. 25, no. 12, pp. 2809–2822, Dec. 2013.
[36] X. Yang, Q. Kuang, W. Zhang, and G. Zhang, "AMDO: An oversampling technique for multi-class imbalanced problems," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 9, pp. 1672–1685, Sep. 2018.
[37] S. Sharma, C. Bellinger, B. Krawczyk, O. Zaiane, and N. Japkowicz, "Synthetic oversampling with the majority class: A new perspective on handling extreme imbalance," in *Proc. IEEE Int. Conf. Data Mining*, 2018, pp. 447–456.
[38] S. Boyd and L. Vandenberghe, *Convex Optimization*. New York, NY, USA: Cambridge Univ. Press, 2004.
[39] C.-L. Liu and P.-Y. Hsieh, "Model-based synthetic sampling for imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 8, pp. 1543–1556, Aug. 2020.
[40] L. Li, H. He, and J. Li, "Entropy-based sampling approaches for multi-class imbalanced problems," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 11, pp. 2159–2170, Nov. 2020.
[41] D. Dua and K. T. Efi, "UCI machine learning repository," Univ. California, Irvine, School of Information and Computer Sciences, 2017. [Online]. Available: http://archive.ics.uci.edu/ml
[42] One-class classifier results, 2005. [Online]. Available: http://homepage.tudelft.nl/n9d04/occ/index.html

Yi Sun is currently working toward the PhD degree in computer science and technology at Hunan University, Changsha, China. His current research interests include imbalanced learning and data mining and machine learning.

Lijun Cai received the PhD degree in computer application technology from Hunan University, Changsha, China. He is currently a full professor of computer science and technology with Hunan University. His research interests include parallel computing, cloud computing, and image processing.

Bo Liao received the PhD degree in computational mathematics from the Dalian University of Technology, Dalian, China, in 2004. From 2004 to 2006, he was a postdoctoral fellow with the University of Chinese Academy of Sciences, Beijing, China. He is currently a full professor with Hainan Normal University. He has authored more than 100 papers in international conferences and journals. His research interests include image processing, bioinformatics, and big data processing.



Wen Zhu received the MS degree in computer science and technology from Hunan University, Changsha, China, in 2010. She is currently a lecturer with Hainan Normal University. Her research interests include image processing and analysis and bioinformatics.

Junlin Xu is currently working toward the PhD degree in computer science and technology at Hunan University, Changsha, China. His research interests include bioinformatics, machine learning, biochemical research method, disease-associated non-coding RNAs, and single cell.